IBM

IBM **Aspera**®

IBM Aspera on Cloud Automation Development Guide

Laurent Martin, IBM

2026/11/08



BUILD • AUTOMATE • ORCHESTRATE

Contents

1 Concepts **3**
- 1.1 Introduction . . . 3
  - 1.1.1 Where to start . . . 3
  - 1.1.2 Typographic conventions . . . 3
- 1.2 How AoC Automation works . . . 4
- 1.3 Terminology . . . 4
- 1.4 States and statuses . . . 5
  - 1.4.1 Workflow states . . . 5
  - 1.4.2 Workflow instance statuses . . . 5

2 Triggers **6**
- 2.1 Manual trigger . . . 6
- 2.2 Scheduled trigger . . . 6
- 2.3 Watch folder trigger . . . 6
- 2.4 Shared inbox trigger . . . 7
- 2.5 Delete folder trigger . . . 7
- 2.6 Trigger conditions and filtering . . . 7

3 Actions **8**
- 3.1 File transfer . . . 8
- 3.2 Package transfer . . . 9
- 3.3 REST API call . . . 9
- 3.4 Email notification . . . 9
- 3.5 Folder creation . . . 9
- 3.6 Custom variables . . . 10
- 3.7 Telestream quality control . . . 10
- 3.8 Action timeout defaults . . . 10

4 Substitution variables **11**
- 4.1 Global workflow instance variables . . . 11
- 4.2 Action output references . . . 12
- 4.3 Path-based substitutions schema . . . 12

5 Managing workflows and jobs **13**
- 5.1 Workflow lifecycle . . . 13
- 5.2 Job lifecycle . . . 13

6 Managing workflows with ascli **14**
- 6.1 CLI commands reference . . . 14
- 6.2 Exporting and importing workflows . . . 14
  - 6.2.1 Exporting a workflow . . . 14
  - 6.2.2 Portable manifest format example . . . 14
  - 6.2.3 Importing a workflow . . . 15

7 Configuring API callback **16**

8 Design recipes and patterns **17**
- 8.1 Watch folder to multi-destination transfer . . . 17
- 8.2 Shared inbox package to MAM webhook . . . 17

8.3 Scheduled sync with differential overwrite 17

9 Best practices and troubleshooting **19**
9.1 Practices checklist 19
9.2 Common pitfalls and resolutions 19

IBM Aspera on Cloud Automation Development Guide

> **Important**
>
> This guide covers workflow development and automated operations using the **IBM Aspera on Cloud (AoC) Automation API** and the **IBM Aspera CLI (`ascli`)**. It applies to IBM Aspera on Cloud Automation API 1.0.5 and higher.

Chapter 1

Concepts

This chapter explains the execution model of AoC Automation, its terminology, and the states of workflows and jobs.

1.1 Introduction

IBM Aspera on Cloud (AoC) Automation provides cloud-native workflow orchestration for fast, reliable, and secure data movement across hybrid and multi-cloud storage tiers.

It is designed for workflow authors, systems integrators, and AI assistants building automated file pipelines with IBM Aspera on Cloud.

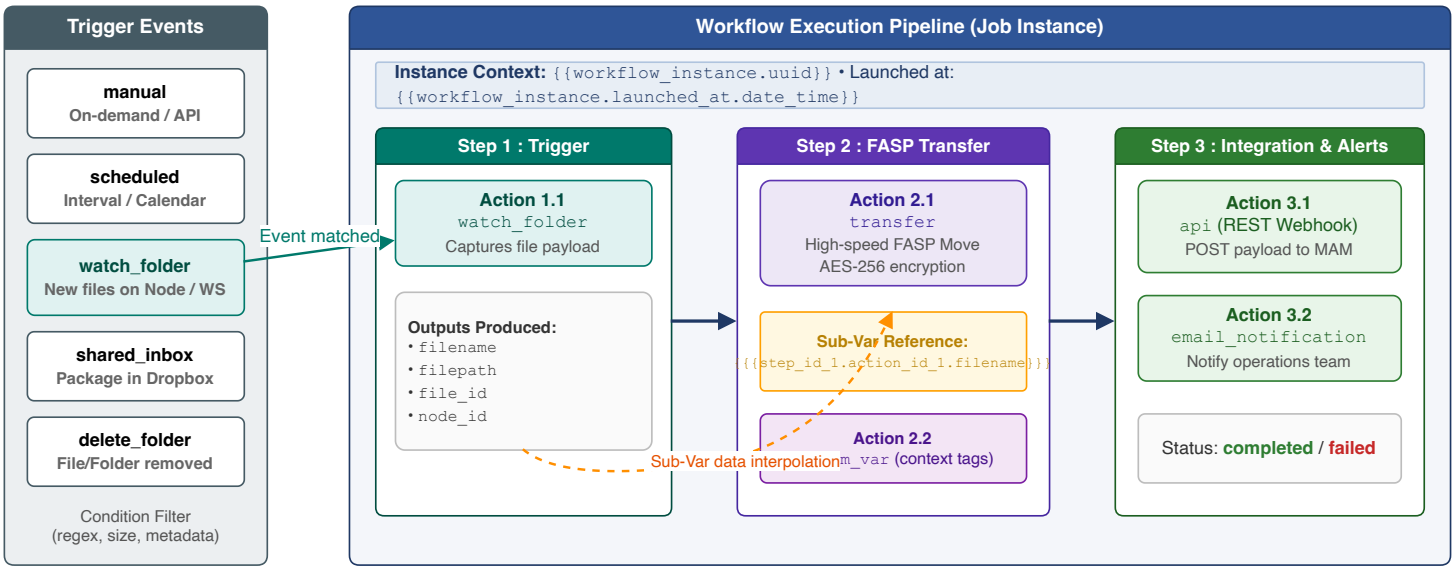
> **Note**
>
> The Automation app is available on the **Standard Plus** and **Premium** editions of Aspera on Cloud. Concurrent job limits: **5** (Standard Plus) / **10** (Premium). Queue size limits: **5,000** (Standard Plus) / **10,000** (Premium). It is not enabled by default: an org admin must enable it under **Admin > Applications > Automation** and add members.

1.1.1 Where to start

Goal	Read
Understand the AoC execution model	How AoC Automation works, Terminology, States and statuses
Configure event and schedule triggers	Triggers, Trigger conditions and filtering
Configure transfers, API calls and actions	Actions
Pass data dynamically between steps	Substitution variables
Manage workflows from the command line	Managing workflows with ascli
Implement common end-to-end patterns	Design recipes and patterns
Check requirements before deployment	Practices checklist, Common pitfalls and resolutions

1.1.2 Typographic conventions

Format	Use	Example
Bold	A label of the web interface: button, menu, tab, field.	**Run Now**, **Active**, **Save**
Code	A literal of a file or code: attribute, status, action type, command, JSON/YAML fragment.	`watch_folder`, `transfer`, `completed`
Italic	A term where it is defined, or a workflow/step name chosen by the author.	*Ingest pipeline*, *Archive Drop*
Double quotes	A message or text that AoC displays.	”Package received”
`<PLACEHOLDER>`	A value that the reader replaces.	`<WORKFLOW_ID>`



AoC Automation execution model: Trigger events, sequential Steps and Actions, and Sub-Vars data flow

1.2 How AoC Automation works

A workflow is an ordered sequence composed of **Steps**, each containing one or more **Actions**:

1. **Definition**: An administrator or user defines a workflow with a single **Trigger** followed by one or more sequential **Steps**.
2. **Step Execution**: Steps run sequentially in the order defined by `step_order`.
3. **Action Execution**: Within each step, actions run in the order defined by `action_order`.
4. **Data Propagation**: Actions pass information downstream using mustache-like **Substitution Variables** (`Sub-Vars`), allowing dynamic destination paths, file names, API payloads, or email bodies.
5. **Instance Lifecycle**: When a trigger condition is met (or when triggered manually), AoC creates a **Workflow Instance** (Job) tracked in the Job Queue.

1.3 Terminology

Term	Meaning	In the web interface
Workflow	The definition of an automated business process. Identified by UUID. Contains any number of steps.	Workflow
Workflow Instance	A single execution run of an active workflow (a Job).	Job
Step	A logical stage containing one or more ordered actions. Steps run sequentially; actions within a step run in parallel.	Step
Action	An individual operational task: Transfer, Send a Package, Call an API, Notify, Custom Variables, Create a Folder.	Action
Trigger	The first action of Step 1 that defines the conditions for launching a workflow instance.	Trigger
Sub-Var (Substitution Variable)	A dynamic placeholder resolved at runtime from prior action outputs or global workflow context.	Variable
Job Queue	The list of running and queued jobs. Beyond the concurrent maximum, jobs queue in FIFO order.	Job Queue
Permissions	Sharing rules allowing another user to view the workflow and manage (start, cancel, delete) its jobs.	Share

1.4 States and statuses

1.4.1 Workflow states

- `draft`: The workflow is under configuration. Automatic triggers are disabled.
- `inactive`: The workflow configuration is completed but execution is paused.
- `active`: The workflow is live. Automatic triggers spawn jobs when triggered.

1.4.2 Workflow instance statuses

- `running`: The job is currently executing steps and actions.
- `queued`: The job is waiting to run because the concurrent maximum has been reached.
- `completed`: All steps and actions finished successfully.
- `failed`: An action encountered an unrecoverable error (action timeout or transfer failure).
- `canceled`: The instance was aborted by a user or administrator.

Chapter 2

Triggers

Every workflow requires exactly one trigger defined as the first action of the first step.

2.1 Manual trigger

- Type: `manual`
- Trigger condition: Triggered on-demand via the Web UI (**Run Now**) or through the REST API (`POST /workflows/:id/launch`) and CLI (`ascli aoc automation workflows launch <id>`).
- Outputs: `workflow_instance.uuid`, `workflow_instance.launched_at.*`.
- **Constraint**: A workflow with a manual trigger must contain at least one `transfer` or `package_transfer` action.

2.2 Scheduled trigger

- Type: `scheduled`
- **Constraint**: A workflow with a time-based trigger must contain at least one `transfer` or `package_transfer` action.
- Configuration parameters:
  - `repeat_interval.repeat_type`: `minutes`, `hours`, `days`, `weeks`, `months`.
  - `repeat_interval.repeat_number`: Interval count (e.g., every 2 hours).
  - `repeat_interval.days_of_the_week`: Boolean flags (`monday` through `sunday`).
  - `repeat_interval.day_of_the_month`: Number (1–31) for monthly schedules.
  - `start_date`, `end_date`: ISO 8601 timestamps bounding the schedule.

> **Note**
>
> The recurrence interval begins when the **previous run completes**, not at the scheduled start time. For example: start at 01:00, repeat every 6 h, run takes 2 h → next run starts before 09:00 (3:00 AM + 6 h).

2.3 Watch folder trigger

- Type: `watch_folder` (labeled **File addition event** in the UI)
- Monitored location:
  - `source.type`: `node` (requires node secret) or `workspace`.
  - `source.node_id` or `source.workspace_id`.
  - `source.file_id`: Root folder ID to monitor.
  - `source.paths`: Monitored sub-paths.
- Condition filter (`trigger_when`): **Any file is added** (default) or **Custom** (name, size, type conditions with `add_condition` list).
- Post-processing archive settings:
  - `source_archive.archive_option`: `none`, `archive` (move within same node), or `delete`.

  - `source_archive.move_after_transfer`: Target directory path on the same node.
  - `source_archive.remove_empty_directories`: Boolean.

2.4 Shared inbox trigger

- Type: `shared_inbox` (labeled **Shared inbox event** in the UI)
- Monitored location:
  - `source.inbox_id`: UUID of the Aspera on Cloud Shared Inbox (Dropbox).
- Condition filter: **Any file is added** (default) or **Custom** — one or more metadata field conditions (field, operator, value). Multiple conditions use `join_type: and | or`.
- Metadata extraction: Outputs package metadata fields filled by the package sender (accessible as Sub-Vars in downstream actions).

2.5 Delete folder trigger

- Type: `delete_folder` (labeled **File or folder deletion event** in the UI)
- Configuration:
  - `trigger_on_file_deletes`: Boolean — trigger when a file matching conditions is deleted.
  - `trigger_on_folder_deletes`: Boolean — trigger when the monitored folder itself is deleted.
  - `source.access_key`: Node access key for delete event monitoring.

2.6 Trigger conditions and filtering

Watch folder and shared inbox triggers support granular condition lists:

```yaml
conditions:
  - field_name: "file_name"
    field_type: "string"
    match_type: "regex"          # includes | excludes | matches | regex
    value: ".*\\.mp4$"
  - field_name: "file_size"
    field_type: "byte"
    match_type: "greater_than"   # greater_than | less_than | equal
    value: 1048576               # bytes (> 1 MB)
join_type: "and"                 # and | or
```

Chapter 3

Actions

Each action step performs one specific task. Multiple actions can run **in parallel** within the same step (click the **+** icon below an existing action in the UI). Available action types: `transfer`, `package_transfer`, `api`, `email_notification`, `custom_var`, `folder`.

3.1 File transfer

Transfers files and directories between nodes, workspaces, or cloud buckets using IBM Aspera high-speed FASP protocol.

```yaml
type: transfer
data:
  source:
    type: node                  # node | workspace; use "same_as_trigger" in the UI to inherit
↪  trigger source
    node_id: "00000000-0000-0000-0000-000000000001"
    file_id: "12345"
  destination:
    type: workspace
    workspace_id: "00000000-0000-0000-0000-000000000002"
    file_id: "67890"
    create_folder: true         # auto-create destination folder if absent (supports Sub-Vars in
↪  folder name)
  rate_policy: high             # high | fair | low | fixed (the more restrictive of node and
↪  workflow settings applies)
  target_rate:
    rate: 100000                # Kbps (100 Mbps) — the lower of node setting and this value is used
    allow_retry: true
    retry_attempts: 3
    retry_interval: 10          # seconds
  resume_policy:
    resume_partials: true
    difference_check: attrs
    destination_file_options: overwrite_newer # overwrite_always | overwrite_never | overwrite_newer
↪  | overwrite_different
  content_encryption:
    encryption: aes-256         # none | aes-128 | aes-192 | aes-256; the higher key-length of node
↪  vs. workflow wins
  preserve_file_attributes: true  # preserve source timestamps at destination
  source_archive:               # post-transfer source handling (node source only)
    archive_option: archive     # none | archive | delete
    move_after_transfer: "/archive"  # archive destination (must be on the same node)
    delete_remaining: false     # delete skipped files (already at destination)
    delete_empty_folders: false # delete empty nested folders after transfer
    delete_empty_source_folder: false  # delete the source folder itself if empty after transfer
```

! Caution

When `delete_empty_source_folder` is enabled and the workflow has multiple transfer actions, ensure each action uses a **different** source folder to avoid unintended deletions.
`rate_policy` and `target_rate` interact with the node settings: the actual behavior always obeys the **more restrictive** setting.

3.2 Package transfer

Sends digital packages using Aspera Packages application.

```yaml
type: package_transfer
data:
  package:
    workspace_id: "00000000-0000-0000-0000-000000000002"
    name: "Automated Package Delivery"
    note: "Package generated by AoC Automation"
    content_retention_duration: 30 # days
    recipients:
      - id: "user_uuid_here"
        type: User
      - id: "group_uuid_here"
        type: Group
```

3.3 REST API call

Executes HTTP REST requests against internal or third-party web services with complete payload interpolation.

```yaml
type: api
data:
  method: post                  # get | post | put | delete
  endpoint: "https://api.example.com/v1/ingest"
  authentication:
    type: bearer                # none | bearer | basic
    token: "YOUR_API_TOKEN"
  headers:
    - key: "Content-Type"
      value: "application/json"
  payload: '{"job_id": "{{workflow_instance.uuid}}", "file": "{{{step_id_1.action_id_1.filename}}}"}'
  verify_ssl: true
  callback: false               # set true to enable API callback (see Configuring API callback)
  timeout: 30000                # milliseconds (default 5 min without callback, 24 h with callback)
```

3.4 Email notification

Dispatches formatted notification emails on workflow milestones.

```yaml
type: email_notification
data:
  subject: "Workflow Complete: {{{step_id_1.action_id_1.filename}}}"
  message: "The file was successfully processed and moved.\nJob UUID: {{workflow_instance.uuid}}"
  recipient_hashes:
    - id: "user_uuid_1"
      type: User
    - id: "qa_team@example.com"
      type: Email
```

3.5 Folder creation

Dynamically creates directory trees on target nodes prior to file transfer.

```yaml
type: folder
data:
  folder_name: "{{workflow_instance.launched_at.YYYY}}-{{workflow_instance.launched_at.MM}}"
  parent_folder:
    type: node
    node_id: "00000000-0000-0000-0000-000000000001"
    file_id: "root_folder_id"
```

3.6 Custom variables

Instantiates named key-value pairs stored in workflow context for downstream consumption. Values can combine literal text, global variables (`{{workflow_instance.uuid}}`), and Sub-Vars from previous steps.

> **Important**
>
> Custom variable names **cannot contain spaces**.

3.7 Telestream quality control

Integrates with Telestream cloud services for automated media quality verification.

3.8 Action timeout defaults

If the Automation app does not receive an update from the node within the configured timeout, the action fails and the job fails.

Action	Default	Minimum	Maximum
`transfer`	24 h	1 h	48 h
`package_transfer`	24 h	1 h	48 h
`api` (no callback)	5 min	1 min	24 h
`api` (with callback)	24 h	1 h	48 h
`email_notification`	5 min	1 min	60 min
`custom_var`	5 min	1 min	60 min
`folder`	5 min	1 min	60 min

Chapter 4

Substitution variables

Substitution variables (Sub-Vars) let workflow actions consume dynamic values at runtime — from the global job context or from outputs of preceding actions. They use mustache-style double or triple curly braces: `{{global}}` for global variables, `{{{step.action.field}}}` for action outputs.

4.1 Global workflow instance variables

These variables are available in every workflow action regardless of trigger type.

Variable Syntax	Description	Example Value
`{{workflow_instance.uuid}}`	Unique identifier (UUID) of the current job instance.	`a1b2c3d4-e5f6-7890-abcd-ef1234567890`
`{{workflow_instance.launched_at.YYYY}}`	4-digit launch year in UTC.	`2026`
`{{workflow_instance.launched_at.MM}}`	2-digit launch month in UTC.	`11`
`{{workflow_instance.launched_at.MMM}}`	3-letter month name in UTC.	`Nov`
`{{workflow_instance.launched_at.DD}}`	2-digit launch day in UTC.	`08`
`{{workflow_instance.launched_at.hh}}`	2-digit launch hour in UTC (24 h).	`14`
`{{workflow_instance.launched_at.mm}}`	2-digit launch minute in UTC.	`30`
`{{workflow_instance.launched_at.date_time}}`	Full ISO 8601 timestamp in UTC (GMT).	`2026-11-08 14:30:00TZD`

4.2 Action output references

Values produced by previous steps and actions are referenced with triple curly braces:

`{{{step_id_<STEP_ID>.action_id_<ACTION_ID>.<FIELD>}}}`

Output Field	Produced By	Description
`filename`	`watch_folder`, `transfer`	Base name of the file or folder.
`filepath`	`watch_folder`, `transfer`	Full source path of the file.
`parent_path`	`watch_folder`	Path to the parent folder.
`file_ext`	`watch_folder`	Extension without leading dot (e.g. `mxf`, `mp4`).
`file_id`	`watch_folder`, `folder`, `transfer`	Aspera node file identifier.
`node_id`	`watch_folder`, `transfer`	Node UUID.
`transfer_uuid`	`transfer`	Transfer session UUID.
`response_code`	`api`	HTTP status code (e.g. `200`, `201`).
`response_body`	`api`	HTTP response body.
`package_id`	`package_transfer`	UUID of the sent package.

4.3 Path-based substitutions schema

In workflow definitions, substitutions are mapped explicitly using the `substitutions` array in action `data`:

```
{
  "substitutions": [
    {
      "template": "Archive/{{{step_id_step1.action_id_act1.filename}}}",
      "path": [
        {"name": "destination", "type": "map"},
        {"name": "paths", "type": "array"},
        {"index": 0, "type": "input_map"},
        {"name": "destination", "type": "map"}
      ]
    }
  ]
}
```

Chapter 5

Managing workflows and jobs

The **Workflows** page lists all your workflows and their current state. The **Job Queue** page shows all running and queued jobs across your workflows. The actions below are available from the overflow menu (⋮) on each row.

5.1 Workflow lifecycle

From the overflow menu (⋮) next to a workflow name in the Workflows page:

Action	Effect	New Status
Activate	Workflow starts launching jobs automatically (event triggers) or on **Run Now** (manual).	`active`
Deactivate	Pauses new jobs; running jobs can complete or be cancelled. Org admins can deactivate other users' workflows.	`inactive`
Edit	Cancels all current and future jobs; puts workflow back into edit mode.	`draft`
Duplicate	Creates a copy in `draft` state. Org admins can duplicate other users' workflows.	`draft`
Delete	Cancels all current and future jobs; removes workflow from the list.	—
Run Now	Manually launches one job instance (any trigger type).	job `running`
Share	Grants another user **Manage jobs** permission (view, start, cancel, delete jobs).	no change
Rename	Renames the workflow; no operational effect.	no change
Email notifications	Configures per-event email alerts (started, completed, failed) for the workflow.	no change
Job queue	Jumps to the job queue filtered to this workflow.	no change

> **Note**
>
> Org admins can deactivate and duplicate workflows created by other users.

5.2 Job lifecycle

From the **Job Queue** page (check the job row, then click the action):

Action	Effect	New Status
Cancel running jobs	Sends an async cancel to the node for in-progress transfers; reports job as `cancelled`. All queued jobs are cancelled automatically.	`cancelled`
Delete jobs	Removes the job from the list.	—

Job step indicators: **green** = success, **blue** = in progress, **red** = failed.

Chapter 6

Managing workflows with ascli

`ascli` covers the full Automation lifecycle: CRUD on workflows, state transitions, job management, export/import for version control, and permissions. All commands start with `ascli aoc automation`.

6.1 CLI commands reference

`ascli` provides full command-line lifecycle management for AoC Automation:

```
# Workflows CRUD and operations
ascli aoc automation workflows list [--query.show_org_workflows=@val:true]
ascli aoc automation workflows show <workflow_id>
ascli aoc automation workflows details <workflow_id>
ascli aoc automation workflows create @json:'{"name":"My Workflow"}'
ascli aoc automation workflows update_state <workflow_id> @json:'{"state":"active"}'
ascli aoc automation workflows delete <workflow_id>
ascli aoc automation workflows duplicate <workflow_id> [--name="New Name"]
ascli aoc automation workflows export <workflow_id>
ascli aoc automation workflows import @yaml:my_workflow.yml

# Execution and instances
ascli aoc automation workflows launch <workflow_id>
ascli aoc automation workflows cancel_instances <workflow_id>
ascli aoc automation instances list [--query.workflow_id=<workflow_id>]
ascli aoc automation instances show <instance_id>
ascli aoc automation instances cancel <instance_id>

# Permissions and sharing
ascli aoc automation workflows permissions <workflow_id> list
ascli aoc automation workflows permissions <workflow_id> create
 ↪  @json:'{"user_id":"...","manage_jobs":true}'
```

6.2 Exporting and importing workflows

Workflows can be fully exported to portable YAML/JSON files for Git version control and CI/CD promotion across environments.

6.2.1 Exporting a workflow

```
ascli aoc automation workflows export 00000000-0000-0000-0000-000000000001 --format=yaml >
 ↪  workflow_backup.yml
```

6.2.2 Portable manifest format example

```
name: "Ingest and Distribute Video"
trigger_type: "watch_folder"
allow_notifications: true
```

```yaml
workflow_notifications:
  failed:
    - id: "admin@example.com"
      type: "Email"
steps:
  - ref_id: "step_1"
    name: "Watch Drop Folder"
    actions:
      - ref_id: "act_trigger"
        name: "Watch Drop Folder"
        type: "watch_folder"
        data:
          source:
            type: "node"
            node_id: "11111111-1111-1111-1111-111111111111"
            file_id: "100"
          conditions:
            - field_name: "extension_name"
              field_type: "string"
              match_type: "matches"
              value: "mp4"
  - ref_id: "step_2"
    name: "Transfer to Cloud Archive"
    actions:
      - ref_id: "act_transfer"
        name: "FASP Transfer"
        type: "transfer"
        data:
          source:
            type: "node"
            node_id: "11111111-1111-1111-1111-111111111111"
            file_id: "100"
          destination:
            type: "workspace"
            workspace_id: "22222222-2222-2222-2222-222222222222"
            file_id: "200"
          resume_policy:
            destination_file_options: "overwrite_newer"
          substitutions:
            - template: "Archives/{{{step_id_step_1.action_id_act_trigger.filename}}}"
              path:
                - name: "destination"
                  type: "map"
                - name: "path"
                  type: "map"
```

6.2.3 Importing a workflow

```
ascli aoc automation workflows import @yaml:workflow_backup.yml
```

`ascli` automatically creates the workflow, steps, and actions in sequence and remaps all internal substitution variable IDs.

Chapter 7

Configuring API callback

When `callback: true` is set on an `api` action, after sending its HTTP request the workflow **pauses** and waits for the external service to call back.

Prerequisites — your API server must: 1. Include the **static callback endpoint** `https://api.ibmaspera.com/automation/v1/external/api_callback` in its response. 2. Echo back the **token** that AoC injects into the payload (parse it from the incoming request body).

Enabling in the UI: 1. Open or create a workflow → select the **Call an API** action. 2. Set **Request type** to `POST` or `PUT`. 3. Check **Add an API callback**. 4. Save and activate the workflow.

Sequence of events (once the action is triggered):

1. AoC sends the configured JSON payload to your API, with a unique `token` field injected automatically. The workflow pauses (default timeout: **24 h**).
2. *(Optional)* Your service may send **in-progress** status notifications to the callback endpoint:

   ```
   { "token": "<TOKEN>", "status": "in_progress", "progress": 42 }
   ```

   Progress (0–100) is displayed in the Automation UI.
3. When your service finishes, it sends a **completion** response:

   ```
   { "token": "<TOKEN>", "status": "success" }
   ```

   or `"status": "failed"` to fail the job.
4. On `success`, the workflow proceeds to the next step.

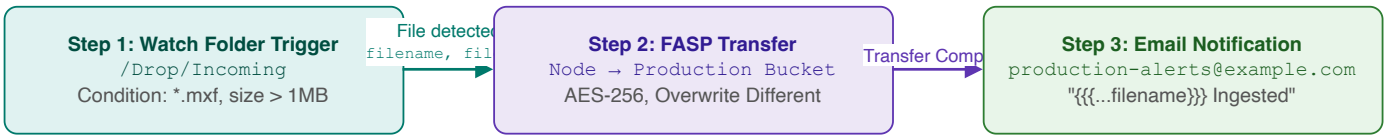
Chapter 8

Design recipes and patterns

Common end-to-end workflow patterns combining triggers, transfers, API calls, and notifications. Each recipe is illustrated with a diagram showing the data flow.

8.1 Watch folder to multi-destination transfer

Goal: Automatically ingest incoming media files, replicate them to production storage with AES-256 encryption, and notify the production team.



Watch folder trigger to multi-destination transfer and notification

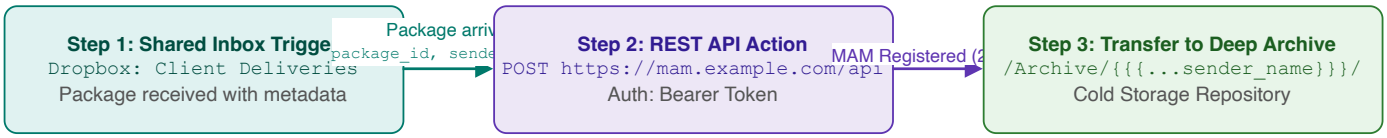
---

8.2 Shared inbox package to MAM webhook

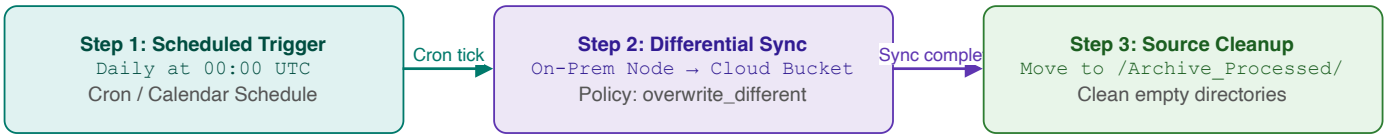
Goal: Listen for client deliveries in a Shared Inbox (Dropbox), notify an external MAM system via REST API, and transfer the payload into deep archive.

8.3 Scheduled sync with differential overwrite

Goal: Run a nightly differential sync every midnight from an on-premise node to cloud storage, then move processed files to an archive folder.



Shared inbox package reception to MAM webhook and deep archive



Scheduled sync with differential overwrite and archive cleanup

Chapter 9

Best practices and troubleshooting

Guidelines to build reliable production workflows, and a table of common failure modes with their root causes and resolutions.

9.1 Practices checklist

- [ ] **Trigger Isolation**: Keep the trigger step dedicated solely to event capture. Perform heavy transfer or API operations in subsequent steps.
- [ ] **Prevent Trigger Loops**: When configuring a `watch_folder` trigger with destination on the same node, always configure `source_archive` (move to archive or delete) or ensure destination paths are excluded from watch rules.
- [ ] **Transfer Idempotency**: Use `overwrite_newer` or `overwrite_different` rather than `overwrite_always` to prevent unnecessary bandwidth consumption during retries.
- [ ] **Secrets Handling**: Do not hardcode static credentials in API action bodies. Use bearer tokens or scoped access keys.
- [ ] **Error Notifications**: Always define `workflow_notifications.failed` recipients on production workflows so the operations team is alerted immediately on failures.

9.2 Common pitfalls and resolutions

Issue	Root Cause	Resolution
Workflow cannot be activated	Missing trigger action, or a step with no actions, or `manual`/`scheduled` trigger with no transfer action.	Ensure Step 1 has a valid trigger; every step has at least one action; `manual` and `scheduled` triggers include a `transfer` or `package_transfer` action.
Cannot edit workflow	Workflow has active or running jobs in the Job Queue.	Run `ascli aoc automation workflows cancel_instances <id>` or **Deactivate** from the UI (running jobs can then complete), then **Edit**.
Substitution variable not resolving	Target step or action ID mismatch, or referencing a downstream step.	Verify that Sub-Vars only reference steps positioned **above** the current step in `step_order`.
Transfer action fails on permission	Aspera on Cloud service user lacks write permissions on destination node.	Ensure the node access key configured in AoC has read/write privileges on the destination storage path.
Transfer files are not re-transferred	Files already present at the destination are skipped (resume policy).	Use `overwrite_always` to force re-transfer, or `overwrite_newer`/`overwrite_different` for differential sync.

Issue	Root Cause	Resolution
Cancel does not stop the node transfer	Automation sends an async cancel; the node may not honour it immediately.	The job is reported `cancelled` by Automation even if the underlying FASP session is still active on the node. Monitor the node logs if immediate cancellation is critical.
Job stays in `queued` indefinitely	Concurrent job maximum reached (5 Standard Plus, 10 Premium).	Wait for running jobs to complete, cancel them from the Job Queue, or upgrade the subscription edition.
API callback never received	External server not configured to call back, or wrong token.	Verify that the server POSTs to `https://api.ibmaspera.com/automation/v1/external/api_callback` with the original `token` value and `"status": "success"` or `"failed"`.

IBM Aspera®

End of document